Mash DB - Feature Roadmap

Current Features (Implemented) ■

Core Database Features

- **CRUD Operations**: INSERT, SELECT, UPDATE, DELETE
- **B-Tree Indexing**: Efficient indexing on id, username, and email
- **Disk Persistence**: Binary serialization with automatic save
- **Paged Storage**: Memory management with page-based architecture
- **SQL Parser**: Token-based parser supporting various SQL commands

Query Capabilities

- **SELECT Queries**:
- `SELECT *` - Select all columns
- `SELECT column1, column2` - Select specific columns
- `SELECT WHERE` - Shorthand for SELECT * WHERE
- **`ORDER BY`** ■ - Sort results by column (ASC/DESC)
- **`LIMIT`** ■ - Restrict result count
- **`OFFSET`** ■ - Skip first N rows for pagination
- **`DISTINCT`** ■ - Remove duplicate rows from results

- **WHERE Clause Operators**:
- Equality: `=`, `!=`
- Comparison: `>`, `<`, `>=`, `<=`
- Logical: `AND`, `OR`
- Multiple conditions with operator precedence

- **INSERT Formats**:
- Simple: `INSERT id username email`
- Full SQL: `INSERT INTO table VALUES (id, 'username', 'email')`

- **UPDATE Operations**:
- `UPDATE table SET column = value WHERE id = n`

- **DELETE Operations**:
- `DELETE WHERE id = n` - Delete by id

- `DELETE WHERE column = value` - Delete by any column
- `DELETE ALL` - Clear entire table

Technical Features

- **Multi-Index Support**: Simultaneous indexing on multiple columns
- **Input Flexibility**: Supports both quoted strings and unquoted identifiers
- **Error Handling**: Comprehensive error messages
- **Data Validation**: Column size limits and duplicate ID prevention
- **REPL Interface**: Interactive command-line interface
- **Result Sorting**: Efficient in-memory sorting for ORDER BY
- **Pagination Support**: LIMIT/OFFSET for result set pagination

---

Planned Features - Phase 1 (Core Enhancements)

1. Advanced Indexing

- **Composite Indexes**: Index on multiple columns (e.g., username + email)
- **Index Statistics**: Track index usage and performance
- **Index Optimization**: Automatic index rebalancing
- **Priority**: HIGH
- **Estimated Effort**: 2-3 weeks

2. Transaction Support

- **ACID Compliance**: Atomic, Consistent, Isolated, Durable operations
- **BEGIN/COMMIT/ROLLBACK**: Transaction control
- **Write-Ahead Logging (WAL)**: Crash recovery
- **Priority**: HIGH
- **Estimated Effort**: 3-4 weeks

3. Enhanced Query Features ■ COMPLETED

- ■ **ORDER BY**: Sort results by columns
- `SELECT * WHERE id > 1 ORDER BY username ASC`
- Supports ASC and DESC for ascending/descending order
- Works with all column types (id, username, email)

- ■ **LIMIT/OFFSET**: Pagination support
- `SELECT * LIMIT 10 OFFSET 20`
- Enables efficient pagination for large result sets
- Can be combined with ORDER BY for sorted pagination
- ■ **DISTINCT**: Remove duplicate results
- `SELECT DISTINCT username`
- `SELECT DISTINCT * WHERE id > 2`
- Uses HashSet for efficient deduplication
- Works with all columns and can be combined with WHERE, ORDER BY, LIMIT
- **Status**: ■ COMPLETED
- **Details**:
- Parser extended with Order, By, Asc, Desc, Limit, Offset, Distinct tokens
- Statement enum includes distinct boolean and optional order_by, limit, offset fields
- Implemented apply_sorting(), apply_offset_limit(), and apply_distinct() helper functions
- All 43 unit tests passing (27 parser + 17 table tests)
- Full integration testing complete
- 5 comprehensive parser tests covering all ORDER BY/LIMIT/OFFSET combinations
- All 39 unit tests passing (17 parser + 17 table + 5 new ORDER BY tests)

Planned Features - Phase 2 (Advanced Queries)

4. JOIN Operations

- **INNER JOIN**: Match rows from multiple tables
- **LEFT/RIGHT JOIN**: Include non-matching rows
- **CROSS JOIN**: Cartesian product
- **Priority**: MEDIUM
- **Estimated Effort**: 3-4 weeks
- **Note**: Requires multi-table support first

5. Aggregate Functions ■ COMPLETED

- ■ **COUNT()**: Count rows
- `COUNT(*)` - Count all rows
- `COUNT(column)` - Count non-null values
- ■ **COUNT(DISTINCT column)** - Count unique values

- ■ **SUM()**: Sum numeric columns
- ■ **AVG()**: Average of numeric columns
- ■ **MIN()/MAX()**: Find minimum/maximum values
- ■ Supports numeric columns (id)
- ■ Supports string columns (username, email) with lexicographic ordering
- ■ **GROUP BY**: Group results by column(s)
- ■ Single column grouping
- ■ Multiple column grouping (composite keys)
- ■ **HAVING**: Filter grouped results
- Supports all aggregate functions in conditions
- Works with COUNT(DISTINCT) and string MIN/MAX
- **Status**: ■ COMPLETED
- **Remaining**: ORDER BY on aggregate columns
- **Details**:
- AggregateColumn enum with Count, Sum, Avg, Min, Max, CountDistinct variants
- Comprehensive aggregate computation with HashSet for distinct counting
- Parser supports all aggregate functions and HAVING clause
- 76 unit tests passing (72 base + 4 COUNT(DISTINCT) tests)
- Full HAVING clause integration with aggregate condition evaluation
- MIN/MAX extended to support string columns with lexicographic comparison

6. Subqueries

- **Nested SELECT**: Queries within queries
- **IN/NOT IN**: Check value in subquery results
- **EXISTS**: Check if subquery returns results
- **Priority**: LOW
- **Estimated Effort**: 2-3 weeks

Planned Features - Phase 3 (Enterprise Features)

7. Multi-Table Support ■ PARTIALLY COMPLETED

- ■ **CREATE TABLE**: Dynamic table creation
- `CREATE TABLE table_name (col1, col2, col3)`

- Creates new table with specified columns
- Stores table registry in HashMap
- Generates .json file for persistence
- ■ **DROP TABLE**: Remove tables
- `DROP TABLE table_name`
- Removes table from registry
- Deletes corresponding .json file
- Error handling for non-existent tables
- [ ] **ALTER TABLE**: Modify table structure
- [ ] **Table Metadata**: Store schema information in catalog
- **Status**: ■ PARTIALLY COMPLETED (CREATE/DROP implemented)
- **Priority**: HIGH (prerequisite for enhanced multi-table operations)
- **Estimated Effort**: 2-3 weeks for remaining features

8. Data Types

- **Type System**:
- INT, BIGINT, SMALLINT
- VARCHAR, TEXT
- BOOLEAN
- DATE, TIMESTAMP
- FLOAT, DOUBLE
- **Type Validation**: Enforce types on insert/update
- **Type Conversion**: Automatic casting where safe
- **Priority**: HIGH
- **Estimated Effort**: 2-3 weeks

9. Constraints

- **PRIMARY KEY**: Enforce unique identifiers
- **FOREIGN KEY**: Referential integrity
- **UNIQUE**: Unique value constraints
- **NOT NULL**: Required fields
- **CHECK**: Custom validation rules
- **DEFAULT**: Default column values
- **Priority**: MEDIUM
- **Estimated Effort**: 2-3 weeks

Planned Features - Phase 4 (Performance & Scale)

10. Query Optimization

- **Query Planner**: Choose optimal execution path
- **Cost-Based Optimization**: Estimate query costs
- **Index Selection**: Automatically choose best index
- **Query Caching**: Cache frequently used queries
- **Priority**: MEDIUM
- **Estimated Effort**: 3-4 weeks

11. Concurrency Control

- **Multi-Version Concurrency Control (MVCC)**: Non-blocking reads
- **Row-Level Locking**: Fine-grained locks
- **Deadlock Detection**: Prevent deadlocks
- **Isolation Levels**: READ COMMITTED, REPEATABLE READ, etc.
- **Priority**: MEDIUM
- **Estimated Effort**: 4-5 weeks

12. Memory Management

- **Buffer Pool**: Intelligent page caching
- **LRU Eviction**: Cache replacement policy
- **Memory Limits**: Configurable memory usage
- **Compression**: Page-level compression
- **Priority**: MEDIUM
- **Estimated Effort**: 2-3 weeks

Planned Features - Phase 5 (Advanced Features)

13. Full-Text Search

- **Text Indexing**: Inverted index for text search
- **LIKE Operator**: Pattern matching

- **MATCH/AGAINST**: Full-text search queries
- **Stemming/Tokenization**: Advanced text processing
- **Priority**: LOW
- **Estimated Effort**: 3-4 weeks

14. Views

- **CREATE VIEW**: Virtual tables
- **Materialized Views**: Cached query results
- **View Updates**: Update base tables through views
- **Priority**: LOW
- **Estimated Effort**: 2-3 weeks

15. Stored Procedures

- **CREATE PROCEDURE**: Define reusable SQL blocks
- **Parameters**: Input/output parameters
- **Control Flow**: IF/ELSE, WHILE loops
- **Priority**: LOW
- **Estimated Effort**: 4-5 weeks

Planned Features - Phase 6 (Ecosystem)

16. Backup & Restore

- **Hot Backup**: Backup without downtime
- **Point-in-Time Recovery**: Restore to specific timestamp
- **Export/Import**: SQL dump and restore
- **Priority**: HIGH
- **Estimated Effort**: 2-3 weeks

17. Client Libraries

- **REST API**: HTTP interface
- **Client SDKs**: Python, Node.js, Rust clients
- **Protocol**: Binary wire protocol for efficiency
- **Connection Pooling**: Manage multiple connections
- **Priority**: MEDIUM

- **Estimated Effort**: 4-5 weeks

18. Monitoring & Tools
- **Performance Metrics**: Query timing, cache hit rates
- **Query Profiler**: EXPLAIN command
- **Admin Tools**: Database inspection utilities
- **Logging**: Comprehensive query and error logging
- **Priority**: MEDIUM
- **Estimated Effort**: 2-3 weeks

Future Considerations

Possible Long-Term Features
- **Replication**: Master-slave replication
- **Sharding**: Horizontal partitioning
- **Clustering**: Multi-node setup
- **Graph Queries**: Graph database capabilities
- **Time-Series Optimization**: Efficient time-series storage
- **Geospatial Data**: Location-based queries
- **JSON Support**: Store and query JSON documents

Implementation Priority Matrix

High Priority (Next 6 months)
1. Transaction Support
2. Multi-Table Support
3. Data Types
4. ORDER BY/LIMIT
5. Backup & Restore

Medium Priority (6-12 months)
1. JOIN Operations

2. Aggregate Functions
3. Constraints
4. Query Optimization
5. Concurrency Control

Low Priority (12+ months)

1. Subqueries
2. Full-Text Search
3. Views
4. Stored Procedures

Contributing

We welcome contributions! Priority areas for community involvement:

- Test coverage improvements
- Documentation enhancements
- Performance benchmarking
- Client library development
- Bug fixes and optimizations

Version History

- **v0.1.0** (Current): Basic CRUD, indexing, WHERE clauses with AND/OR
- **v0.2.0** (Planned): Transactions, ORDER BY, LIMIT
- **v0.3.0** (Planned): Multi-table, JOINs, aggregates
- **v1.0.0** (Target): Production-ready with ACID compliance